

Unit 1: Database Management System

A. What is a File System?

A **File System** is a method used by an operating system to **store, organize, manage, and retrieve data in files** on a storage device such as a hard disk, SSD, pen drive, or memory card.

In a traditional file-based system, data is stored in **separate files**, and application programs are responsible for creating, reading, updating, and deleting those files.

Simple Definition

A File System is an approach in which data is stored and managed as individual files using the operating system's file management facilities.

Real-Life Example

Consider a college that stores student information using files:

College Student Data

- Student.txt
- Fees.txt
- Attendance.txt
- Examination.txt
- Library.txt
- Placement.txt

Different departments or applications may maintain their own files.

For example:

Student.txt

PRN	Name	Course
20230801	Rahul Sharma	BCA
20230802	Priya Patil	BCA
20230803	Amit Kumar	B.Sc.

Fees.txt

PRN	Name	Fees
20230801	Rahul Sharma	85000
20230802	Priya Patil	85000

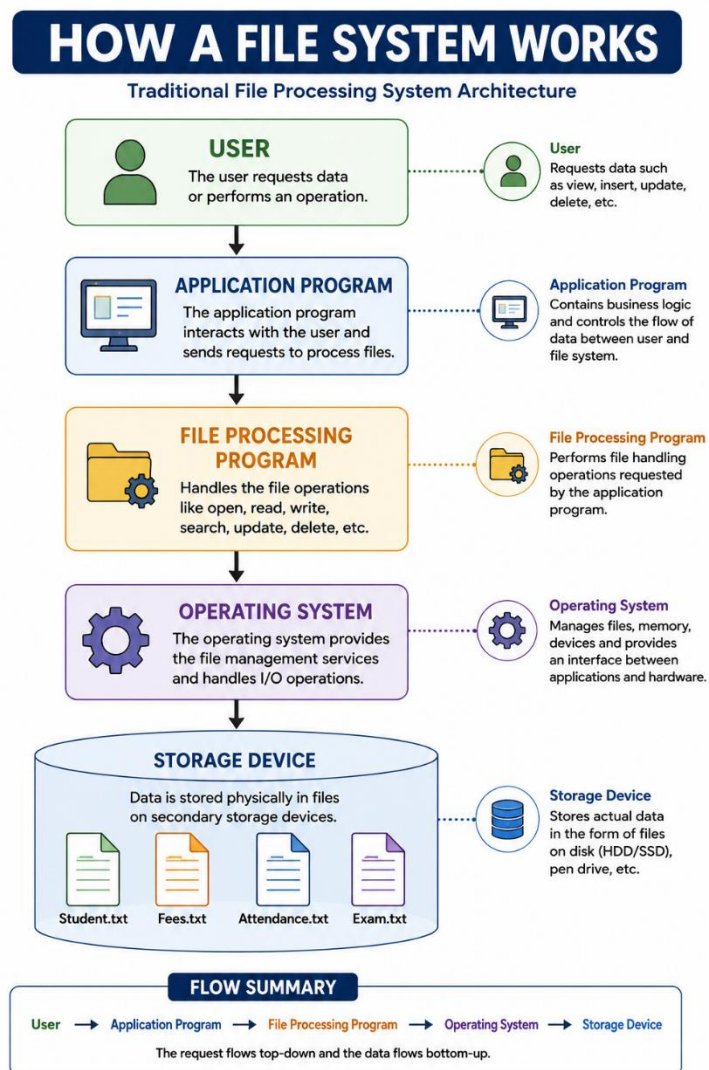
Attendance.txt

PRN	Name	Attendance
20230801	Rahul Sharma	82%
20230802	Priya Patil	91%

Here, the data is stored in **separate files rather than in a centralized database**.

B. How Does a File System Work?

A simple file-based system generally works as follows:



Example

Suppose a student wants to view their fees.

The application may:

1. Open `Fees.txt`
2. Search for the student's PRN
3. Read the corresponding record
4. Display the fees

The application itself contains much of the logic required to **access and manipulate the data**.

C. Advantages of File System

File systems are still useful in many situations, especially for **small and simple applications**.

1. Simple to Understand

Files are relatively easy for beginners to understand.

For example:

```
student.txt
```

can directly contain:

```
101,Rahul,BCA
102,Priya,BCA
103,Amit,BCA
```

There is no need to understand database concepts such as:

- Tables
- Primary Keys
- Foreign Keys
- SQL
- Transactions
- Database schemas

2. Easy to Implement

For a small application, creating a file can be much simpler than setting up a complete DBMS.

For example, a small Python application can create a file:

```
with open("students.txt", "w") as file:
    file.write("101,Rahul,BCA")
```

This requires very little infrastructure.

3. Low Cost

A simple file-based application does not necessarily require a dedicated database server.

For small applications, this can reduce:

- Software requirements
 - Server requirements
 - Administration
 - Maintenance cost
-

4. Suitable for Small Applications

File systems can work well when:

- Data volume is small
- Only one or a few users access the data
- Relationships between data are simple
- Security requirements are limited

Example

A student may maintain personal marks in:

`marks.txt`

For such a small requirement, using a full DBMS may be unnecessary.

5. Easy Backup

Individual files can be copied easily.

For example:

```
student.txt
      ↓
Backup
      ↓
student_backup.txt
```

Files can also be copied to:

- External hard disks
- Pen drives
- Cloud storage
- Another computer

D. Disadvantages of File System

No.	Major Drawback	Key Concept
1	Data Redundancy	Duplicate data
2	Data Inconsistency	Conflicting data
3	Difficult Data Access & Isolation	Difficult searching/integration
4	Integrity Problems	Invalid/incorrect data
5	Atomicity & Recovery Problems	Incomplete updates/data loss
6	Concurrency & Multi-User Problems	Simultaneous access
7	Security Problems	Limited access control
8	Program–Data Dependence	Application depends on file structure

1. Data Redundancy

Meaning

Data redundancy means the same data is unnecessarily stored in multiple files or locations.

Example

Student.txt

101 Rahul Sharma BCA

Fees.txt

101 Rahul Sharma 85000

Attendance.txt

101 Rahul Sharma 82%

The student's name is repeated in several files.

Problems

- Wastes storage space
- Increases maintenance
- Requires multiple updates
- May lead to inconsistency

Remember

Redundancy = Same data stored repeatedly.

2. Data Inconsistency

Meaning

Data inconsistency occurs when **different copies of the same data have different values.**

Example

```
Student.txt      → Rahul Sharma
Fees.txt         → Rahul Kumar
Attendance.txt   → Rahul Sharma
```

Which name is correct?

The system cannot reliably determine the correct value.

Real-Life Example

If a student's name changes from:

```
Rahul Sharma
      ↓
Rahul S. Sharma
```

but only some files are updated:

```
Student.txt      → Rahul S. Sharma
Fees.txt         → Rahul Sharma
Exam.txt         → Rahul Sharma
```

the data becomes inconsistent.

Remember

Inconsistency = Different values for the same data.

3. Difficult Data Access & Data Isolation

Meaning

In a file system, information may be distributed across **multiple files and different formats**, making it difficult to retrieve and combine data.

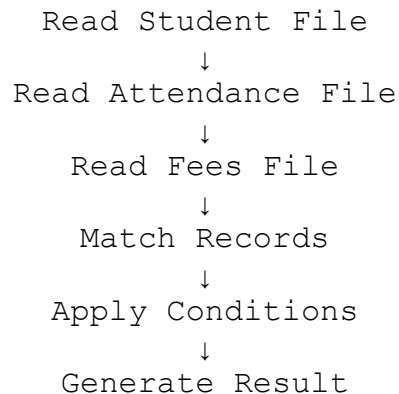
Example

Student.txt	→ Student details
Fees.csv	→ Fee details
Attendance.dat	→ Attendance
Exam.xlsx	→ Examination

Suppose the administrator asks:

Find students having attendance below 75% and pending fees.

The application may need to:



This requires additional programming.

In a DBMS, such requirements can be handled using SQL.

```
SELECT student_name
FROM Student
WHERE attendance < 75
AND fees_status = 'Pending';
```

Remember

Data Isolation = Related data is scattered across different files.

4. Integrity Problems

Meaning

Data integrity means maintaining the **accuracy, validity, and correctness of data**.

Traditional file systems have limited built-in mechanisms for enforcing data rules.

Example

Invalid data:

Age = -25
Attendance = 150%
Marks = 125/100

A DBMS can enforce constraints such as:

PRIMARY KEY
FOREIGN KEY
NOT NULL
UNIQUE
CHECK

For example:

CHECK (attendance >= 0 AND attendance <= 100)

This helps prevent invalid values.

Remember

Integrity = Data should be accurate, valid and correct.

5. Atomicity & Recovery Problems

These two concepts can be taught together because both are related to **system failure during data updates**.

Atomicity

A transaction should be completed **fully or not at all**.

Example: Bank Transfer

Transfer ₹5,000 from Account A to Account B:

Account A → -₹5,000
Account B → +₹5,000

Suppose the system crashes after:

Account A → -₹5,000 ✓
Account B → +₹5,000 ✗

The transaction is incomplete.

Recovery Problem

After a crash, restoring files to a **consistent state** can be difficult.

A DBMS provides mechanisms such as:

- **COMMIT**
- **ROLLBACK**
- **Transaction logs**
- **Recovery mechanisms**

Remember

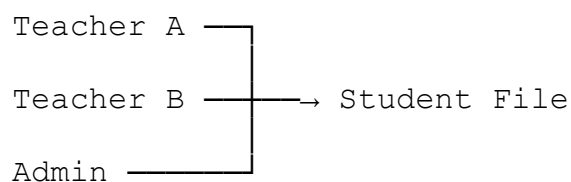
Atomicity = All or Nothing.

6. Concurrency & Multi-User Problems

Meaning

Concurrency occurs when multiple users or applications access or modify the same data simultaneously.

Example



Suppose two teachers update the same student's record at the same time.

This may result in:

- Lost updates
- Incorrect values
- File corruption
- Inconsistent records

Real-Life Example

Two teachers simultaneously update:

Student 101
Attendance = 75%

Teacher A changes it to:

76%

Teacher B changes it to:

78%

One update may overwrite the other.

DBMSs use **concurrency control mechanisms** to manage simultaneous access.

Remember

Concurrency = Multiple users accessing data at the same time.

7. Security Problems

Meaning

Traditional file systems can provide basic file permissions, but **fine-grained control over specific data** can be difficult.

Example

A college may have:

User	Required Access
Student	Own marks
Teacher	Marks of assigned students
HOD	Department records
Administrator	Complete records

In a file-based system, implementing such detailed access control can be difficult.

A DBMS provides mechanisms such as:

- Authentication
- Authorization
- User roles
- Privileges
- Access control

Remember

Security = Only authorized users should access appropriate data.

8. Program–Data Dependence

Meaning

In a traditional file system, application programs are often **dependent on the structure and format of the files**.

Example

Initially:

```
Student.txt
```

```
101,Rahul,BCA
```

The application is designed to read:

```
PRN, Name, Course
```

Later, the file structure changes:

```
101,Rahul,Computer Applications,BCA
```

Now the application may need to be modified because the file structure has changed.

File Structure Changes

↓

Application Program Changes

↓

Additional Development

↓

Higher Maintenance

This is called **program–data dependence**.

A major objective of DBMS is to provide **data independence**.

Remember

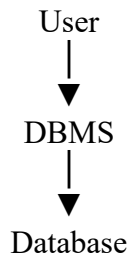
Program–Data Dependence = Changes in data structure may require changes in application programs.

E. What is Database Management System (DBMS)?

Definition:

A Database Management System (DBMS) is software that provides an interface for creating, storing, organizing, retrieving, updating, and managing data in a database.

Basic Structure



The **database** contains the actual data, while the **DBMS** manages access to that data.

Example: College Student Database

Suppose a college needs to store student information.

The database may contain a **Student** table:

PRN	Name	Course	Semester
101	Rahul Sharma	BCA	1
102	Priya Patil	BCA	1
103	Amit Kumar	B.Sc.	1

Here, the table contains the actual student data.

A DBMS such as **MySQL, PostgreSQL, Oracle Database, or Microsoft SQL Server** can be used to create and manage this database.

For example, a user can ask the DBMS to retrieve the details of Rahul:

```
SELECT * FROM Student WHERE PRN = 101;
```

The DBMS processes this request and retrieve the corresponding record:

```
PRN      : 101
Name     : Rahul Sharma
Course   : BCA
Semester : 1
```

Similarly, if the student's course needs to be changed, the DBMS can update the record:

```
UPDATE Student SET Course = 'B.Sc. Computer Science' WHERE PRN = 101;
```

The important point is that the user **does not need to directly manipulate the physical storage files**. The DBMS provides a controlled way to interact with the database.

F. Advantages of DBMS

A DBMS provides several advantages over traditional file-processing systems. For First-Year students, the following eight primary advantages are sufficient and technically important.

No.	Primary Advantage	Simple Meaning
1	Reduced Data Redundancy	Minimizes unnecessary duplicate data
2	Improved Data Consistency	Keeps data accurate and consistent
3	Data Sharing & Centralized Management	Multiple users can access shared data
4	Data Security	Controls who can access data
5	Data Integrity	Maintains accuracy and validity of data
6	Concurrent Access	Supports multiple users accessing data simultaneously
7	Backup & Recovery	Protects data against failures and restores it when required
8	Data Independence	Separates applications from physical data storage

1. Reduced Data Redundancy

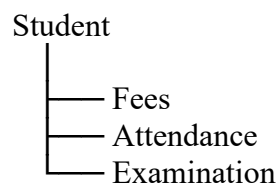
A DBMS reduces unnecessary duplication of the same data.

Example

Instead of storing a student's details repeatedly:

Student.txt
Fees.txt
Attendance.txt
Exam.txt

the DBMS can maintain a central Student record and connect it with other information using a unique identifier such as PRN.



Key idea: Less duplicate data → less storage and maintenance.

2. Improved Data Consistency

Because unnecessary duplication is reduced, the DBMS helps maintain consistent data.

Example

If a student's course changes from:

BCA



B.Sc. Computer Science

the change can be maintained centrally rather than manually modifying several independent files.

Key idea:

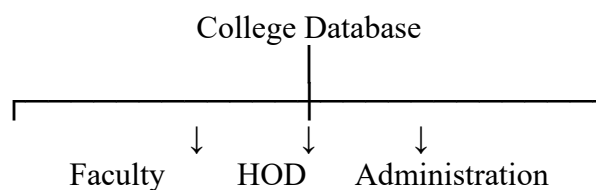
One correct value should be consistently available wherever it is required.

3. Data Sharing & Centralized Management

A DBMS allows multiple authorized users and applications to access the same database.

Example

In a college:



All authorized users can work with the required student information from a centrally managed database.

Key idea:

Centralized data can be shared among authorized users.

4. Data Security

A DBMS provides mechanisms to control access to data.

Example

Consider:

User	Access
Student	View own marks
Faculty	View/update assigned students' marks
HOD	Department records
Administrator	Authorized administrative data

A student should not be allowed to modify another student's marks.

Key idea:

Only authorized users should access or modify appropriate data.

5. Data Integrity

A DBMS helps ensure that stored data is accurate, valid, and follows defined rules.

Example

Attendance should always be between 0% and 100%.

CHECK (attendance >= 0 AND attendance <= 100)

Therefore:

Attendance = 85% ✓

Attendance = 120% ✗

Similarly, DBMS constraints can help ensure that:

- PRN is unique
- Required fields are not empty
- Related records are valid

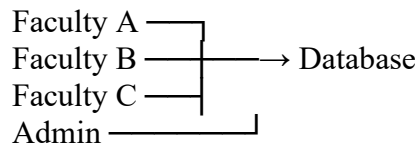
Key idea: DBMS helps prevent invalid data from entering the database.

6. Concurrent Access

A DBMS allows multiple users to access the same database at the same time while managing conflicts.

Example

During examination result processing:



Several authorized users may work with student records simultaneously.

The DBMS uses concurrency control to help prevent problems such as lost updates and inconsistent data.

Key idea:

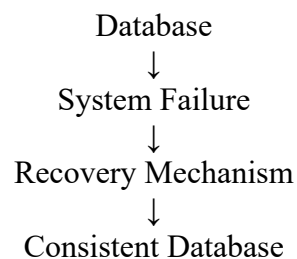
Multiple users can work with shared data safely.

7. Backup & Recovery

A DBMS provides mechanisms for protecting data and recovering it after failures.

Example

Suppose a database server crashes while student records are being updated.



DBMSs can use mechanisms such as:

- Backup
- Transaction logs
- Commit
- Rollback
- Recovery

to restore data to a consistent state.

Key idea: DBMS helps protect data from system failures and data loss.

8. Data Independence

This is one of the most important technical advantages of a DBMS.

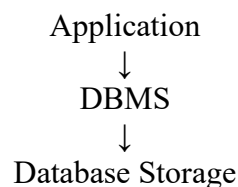
Data independence means that changes in the way data is stored or organized do not necessarily require changes to application programs.

Example

Suppose a college application accesses the student database.

Initially, data may be stored on one type of storage system. Later, the database administrator changes the underlying storage arrangement to improve performance.

The application can continue to work through the DBMS without requiring major changes.



The DBMS provides a layer between the application and physical storage.

Key idea: Applications are separated from the details of physical data storage.

G. Disadvantages of DBMS

Although a DBMS provides many advantages over a traditional file system, it also has some **limitations and disadvantages**. For First-Year students, the following **eight primary disadvantages** are sufficient.

No.	Disadvantage	Meaning	Example
1	High Cost	Requires software, hardware, and infrastructure	Database server and licensed DBMS
2	Complexity	DBMS is more complex than simple files	Need to understand SQL, tables, relationships
3	Requires Skilled Personnel	Specialized people may be required	Database Administrator (DBA)
4	High Resource Requirements	Requires more memory, storage, and processing power	Large DBMS server
5	Performance Overhead	DBMS operations can add processing overhead	Security, transactions, concurrency checks

No.	Disadvantage	Meaning	Example
6	Single Point of Failure	Central database failure can affect many users	Database server goes down
7	Maintenance & Administration	Requires regular monitoring and maintenance	Backup, updates, security, optimization
8	Security Risks	Centralized data can become a valuable target	Unauthorized access or database attack

1. High Cost

A DBMS may require significant investment in:

- Database software
- Servers
- Storage
- Networking
- Backup infrastructure
- Security systems

Some enterprise DBMS solutions may also require licensing costs.

Example

A large organization may need:

Database Software

+

Database Server

+

Storage

+

Backup System

+

Network Infrastructure

For a very small application, a simple file may be much cheaper.

Key idea:

DBMS can be expensive compared with simple file storage.

2. Complexity

A DBMS is more complicated than storing information in a simple text file.

Students and developers may need to understand:

- Tables
- Rows and columns
- Keys
- Relationships
- SQL
- Transactions
- Constraints
- Database design

Example

A simple file can contain:

101,Rahul,BCA

But designing a proper database may require:

Student

Course

Department

Enrollment

and relationships between these tables.

Key idea:

More powerful than files, but also more complex.

3. Requires Skilled Personnel

Large databases often require trained professionals such as a **Database Administrator (DBA)**.

A DBA may be responsible for:

- Database configuration
- User management
- Security
- Backup
- Recovery
- Performance monitoring
- Database maintenance

Example

A bank cannot simply install a DBMS and leave it unmanaged. Someone must continuously monitor the database environment.

Key idea:

Managing a large DBMS requires specialized knowledge and skills.

4. High Resource Requirements

A DBMS generally requires more computational resources than a simple file-processing application.

It may require:

- RAM
- CPU
- Disk storage
- Network resources

The DBMS itself also requires resources for managing:

- Queries
- Transactions
- Security
- Concurrency
- Indexes
- Recovery

Example

Simple File



Low Resource Requirement

DBMS



CPU + RAM + Storage + Database Services

Key idea:

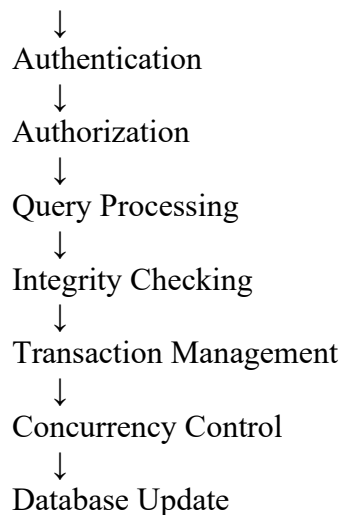
DBMS requires more system resources than basic file processing.

5. Performance Overhead

A DBMS performs many operations to ensure that data is secure, consistent, and reliable.

For example, when updating data, the DBMS may need to perform:

User Request



These operations provide reliability but also introduce **processing overhead**.

Example

For a very small application that only needs to read one simple text file, using a full DBMS may be unnecessary.

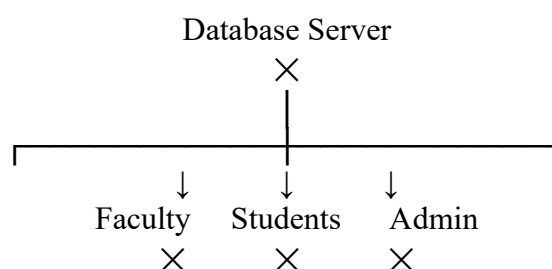
Key idea:

The additional DBMS management mechanisms can introduce processing overhead.

6. Single Point of Failure

In a centralized database environment, many applications may depend on the same database server.

If the central database becomes unavailable:



Many users may lose access to the required data.

Example

If a university's central database server goes down during examination registration, many users may be unable to access the registration system.

Key idea:

Failure of a central database can affect many dependent applications and users.

However, modern systems reduce this risk using **replication, clustering, failover, and backup systems.**

7. Maintenance & Administration

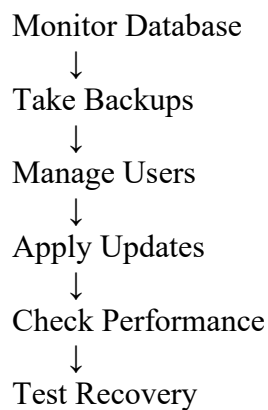
A DBMS requires continuous management.

Regular activities may include:

- Backup
- Software updates
- Security updates
- Performance monitoring
- User management
- Storage management
- Recovery testing

Example

A college database administrator may need to:



Key idea:

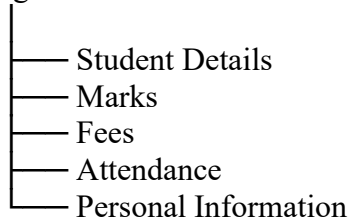
DBMS requires regular administration and maintenance.

8. Security Risks

A centralized database may contain a large amount of valuable information.

For example:

College Database



If proper security controls are not implemented, unauthorized users may gain access to sensitive information.

Possible risks include:

- Unauthorized access
- Data theft
- Accidental modification
- Malware or database attacks
- Loss of confidential information

Example

If an unauthorized user obtains database credentials, they may attempt to access confidential student or employee information.

Key idea:

Centralized data requires strong security controls.

H. Applications of DBMS

A **Database Management System (DBMS)** is used wherever large amounts of structured data need to be **stored, organized, accessed, updated, and managed** efficiently.

For First-Year students, the following **eight major applications** are easy to understand and technically relevant.

No.	Application Area	Data Managed	Example
1	Banking & Finance	Accounts, transactions, loans	ATM and online banking
2	Education	Students, courses, marks, attendance	College Management System
3	Healthcare	Patients, doctors, treatments, reports	Hospital Management System
4	E-Commerce	Products, customers, orders, payments	Online shopping
5	Airline & Railway Reservation	Passengers, schedules, tickets	Ticket booking system

No.	Application Area	Data Managed	Example
6	Library Management	Books, members, issue/return records	College library
7	Telecommunications	Customers, plans, calls, billing	Mobile network
8	Government & Public Services	Citizens, documents, services	Government portals

1. Banking & Finance

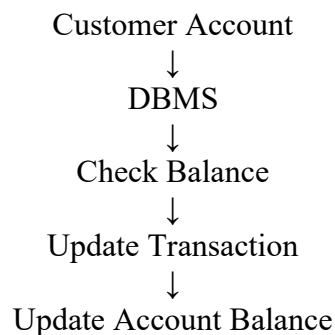
DBMS is extensively used to manage financial information.

Data Managed

- Customer accounts
- Account balances
- Transactions
- Loans
- Credit/debit information
- ATM transactions

Example

When a customer withdraws ₹5,000:



The DBMS ensures that the transaction is properly recorded.

2. Education

Educational institutions use DBMS to manage academic and administrative information.

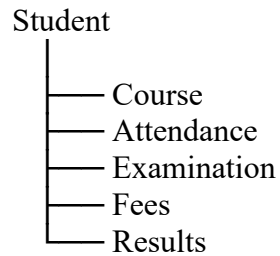
Data Managed

- Student records
- Courses
- Faculty
- Attendance

- Examination
- Marks
- Fees

Example

A college database may contain:



A faculty member can retrieve the required student records from the database.

3. Healthcare

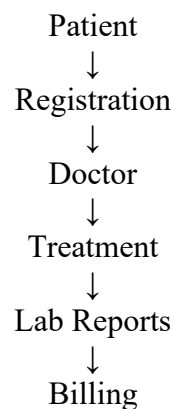
Hospitals and healthcare organizations use DBMS to manage patient and medical information.

Data Managed

- Patient details
- Doctor details
- Appointments
- Medical records
- Laboratory reports
- Medicines
- Billing

Example

When a patient visits a hospital:



All related information can be maintained in a centralized database.

4. E-Commerce

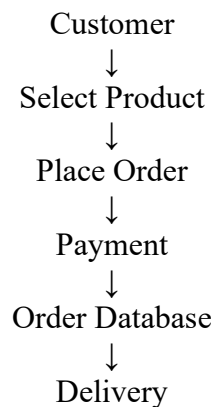
Online shopping platforms use DBMS to manage millions of products, customers, and transactions.

Data Managed

- Customer accounts
- Products
- Product prices
- Orders
- Payments
- Delivery information
- Reviews

Example

When a customer places an order:



The DBMS maintains the information associated with the order.

5. Airline & Railway Reservation

Reservation systems use databases to manage passengers, schedules, seats, and tickets.

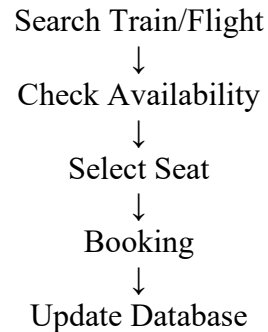
Data Managed

- Passenger information
- Train/flight schedules
- Seat availability
- Booking details
- Cancellation

- Payment information

Example

When a passenger books a seat:



The database must be updated so that the same seat is not assigned to another passenger.

6. Library Management

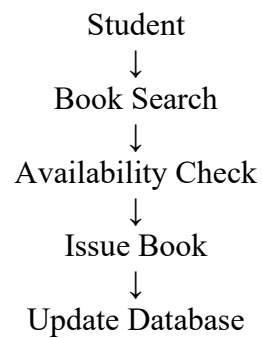
Libraries use DBMS to manage books and members.

Data Managed

- Book information
- Author
- Publisher
- Student/member details
- Issue records
- Return records
- Fine information

Example

When a student issues a book:



The DBMS records that the book is currently issued.

7. Telecommunications

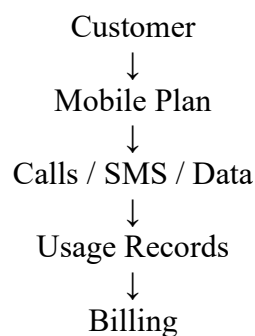
Telecommunication companies use DBMS to manage huge amounts of customer and network information.

Data Managed

- Customer information
- Mobile numbers
- Recharge records
- Plans
- Call records
- Data usage
- Billing

Example

A mobile operator needs to maintain:



A DBMS helps organize and retrieve this information.

8. Government & Public Services

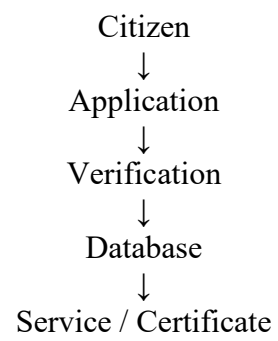
Government organizations use DBMS to manage large-scale citizen and administrative data.

Data Managed

- Citizen records
- Identification information
- Tax records
- Licenses
- Certificates
- Land records
- Public service applications

Example

A government service portal may follow:



The database maintains the information required to process the application.



APPLICATIONS OF DBMS

DBMS is used to store, organize, manage and retrieve large amounts of data efficiently in many real-world applications.



1 BANKING & FINANCE



Data Managed

- Customer accounts
- Transactions
- Loans
- ATM operations
- Account balance

Example

ATM withdrawal, online banking, fund transfer.

2 EDUCATION



Data Managed

- Student records
- Courses
- Attendance
- Examination
- Marks and results
- Fees information

Example

College Management System, student information system.

3 HEALTHCARE



Data Managed

- Patient details
- Doctor information
- Appointments
- Treatment records
- Lab reports
- Billing

Example

Hospital Management System, electronic health records.

4 E-COMMERCE



Data Managed

- Products
- Customers
- Orders
- Payments
- Delivery details
- Reviews

Example

Online shopping, order processing, inventory management.

5 RAILWAY / AIRLINE RESERVATION



Data Managed

- Passenger information
- Schedules
- Seat availability
- Booking details
- Ticket information
- Payment records

Example

Ticket booking system, seat reservation, cancellation.

6 LIBRARY MANAGEMENT



Data Managed

- Book information
- Author & publisher
- Member details
- Issue/return records
- Fine information

Example

Library automation, book issue/return, member management.

7 TELECOMMUNICATIONS



Data Managed

- Customer information
- Mobile numbers
- Plans & services
- Call/SMS/Data records
- Billing information

Example

Mobile network management, recharge, billing, usage tracking.

8 GOVERNMENT & PUBLIC SERVICES



Data Managed

- Citizen records
- Identification documents
- Tax records
- Licenses & certificates
- Land records
- Public service applications

Example

E-governance portals, land records system, permit and license system.



Why DBMS?



Organized Data



Shared Data



Secure Data



Efficient Access



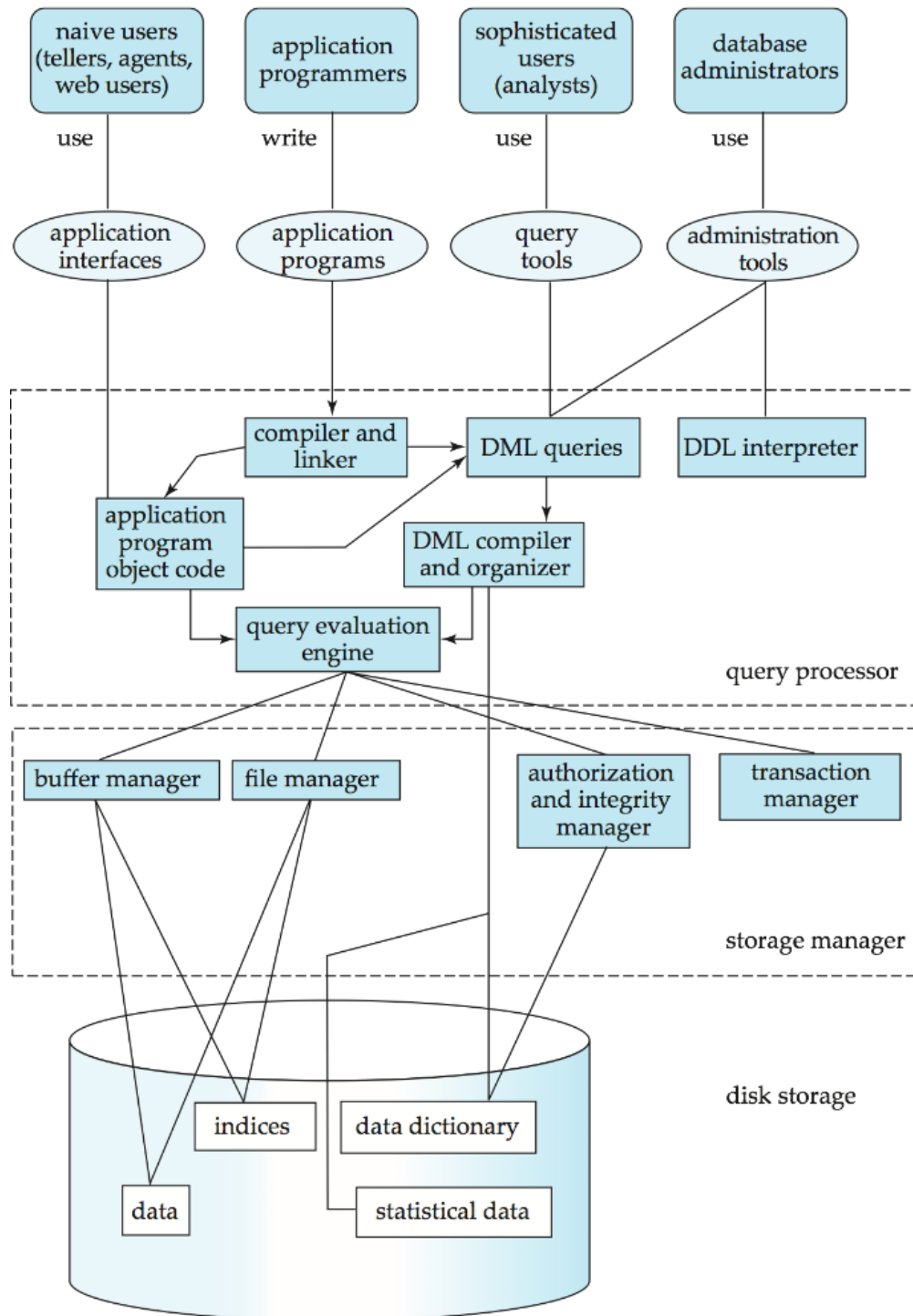
Reliable Management

Result

Better decision making, fast access, accuracy, and data consistency.

Database Architecture:

Database architecture consists of four major sections: **Database Users, Query Processor, Storage Manager, and Disk Storage**. Database Users interact with the database through applications or queries to access and manipulate data. The **Query Processor** interprets and executes users' queries and converts them into instructions that the database can understand. The **Storage Manager** acts as an interface between the query processor and the physical data, managing data storage, retrieval, transactions, and security. Finally, **Disk Storage** contains the actual database files, indexes, data records, and other supporting information stored on physical storage devices.



Types of Database Users

1. Naive User

A **Naive User** is a user who interacts with the database through a predefined application or user interface. They do not need to know how the database is designed or how SQL queries work.

Example: A bank customer using an ATM or a student checking marks through a college portal.

2. Application Programmer

An **Application Programmer** develops software applications that interact with the database. They write programs and SQL queries to insert, update, delete, and retrieve data from the database.

Example: A programmer developing a college management system.

3. Sophisticated User

A **Sophisticated User** directly interacts with the database using SQL or other database tools. They have good knowledge of databases and can write complex queries to analyze and retrieve information.

Example: A data analyst generating reports using SQL queries.

4. Database Administrator (DBA)

A **Database Administrator (DBA)** is responsible for managing and maintaining the database system. The DBA handles **security, user access, backup, recovery, performance, and database maintenance**.

Example: A DBA managing the database servers and controlling access to a university's student database.

Query Processor

The **Query Processor** is a major component of DBMS that receives queries from users or application programs, interprets them, and converts them into instructions that can be executed by the database system. Its main components are:

1. Application Program Object Code

It is the **compiled form of an application program** that contains database access instructions. The DBMS executes these instructions to perform required database operations.

2. Compiler and Linker

The **compiler** converts the application program written in a programming language into object code. The **linker** combines the object code with required libraries and produces an executable program.

3. DML Queries

DML (Data Manipulation Language) queries are used to access and modify data in the database. Common operations include **SELECT, INSERT, UPDATE, and DELETE**.

4. DDL Interpreter

The **DDL (Data Definition Language) Interpreter** processes DDL commands and records the database structure or schema in the system catalog.

Examples: CREATE, ALTER, and DROP.

5. DML Compiler and Organizer

The **DML Compiler** translates DML queries into a low-level query execution plan. The **Query Optimizer** selects an efficient way to execute the query, reducing execution time and resource usage.

6. Query Evaluation Engine

The **Query Evaluation Engine** executes the query execution plan generated by the query processor. It interacts with the **Storage Manager** to retrieve or modify the required data.

Storage Manager

The **Storage Manager** is a component of DBMS that provides an interface between the **database stored on disk and the application programs or queries**. It is responsible for storing, retrieving, and managing data efficiently. Its main components are:

1. Buffer Manager

The **Buffer Manager** manages the transfer of data between **disk storage and main memory**. It loads required data into memory and writes modified data back to disk.

Example: When a query needs a record stored on disk, the Buffer Manager brings that record into memory.

2. File Manager

The **File Manager** manages the allocation of **disk space and database files**. It organizes data files, records, and indexes on storage devices.

Example: It determines where a particular database table or record is stored on disk.

3. Authorization and Integrity Manager

The **Authorization and Integrity Manager** ensures that users have appropriate **access permissions** and that database rules or constraints are maintained.

Example: It prevents an unauthorized user from modifying student records and ensures that a student's PRN is unique.

4. Transaction Manager

The **Transaction Manager** manages database transactions and ensures **consistency and reliability** of the database. It handles transaction control, concurrency, and recovery when failures occur.

Example: In a bank transfer, it ensures that money is deducted from one account and added to another correctly.

Disk Storage

Disk Storage is the physical storage area where the database and related information are permanently stored. It mainly contains the following:

1. Data

Data refers to the **actual records stored in the database**. It includes information such as student details, employee records, customer information, etc.

Example: Student PRN, name, course, marks, and contact details.

2. Indices

Indices are special data structures used to **speed up the retrieval of records** from the database. They help the DBMS locate data quickly without searching every record.

Example: An index created on the PRN field can quickly find a particular student.

3. Data Dictionary

A **Data Dictionary** contains **metadata**, or information about the database structure. It stores details about tables, columns, data types, constraints, relationships, and users.

Example: It may store that the PRN column is an integer and is a primary key.

4. Statistical Data

Statistical Data contains information about the database that helps the DBMS **optimize query execution**. It may include the number of records, number of distinct values, and data distribution.

Example: The DBMS may know that a table contains 1,00,000 records and use this information to select an efficient query execution plan.

What is DBA?

A **Database Administrator (DBA)** is a person responsible for **managing, maintaining, securing, and controlling a database system**. The DBA ensures that the database is **secure, reliable, consistent, and available** to authorized users.

Major Roles of DBA

1. Database Security and Authorization

The DBA controls user access and permissions to protect the database from unauthorized access.

Example: A DBA gives **read-only access** to students while providing full access to authorized administrative staff.

2. Database Backup and Recovery

The DBA performs regular backups and restores the database in case of data loss, system failure, or corruption.

Example: If a database server crashes, the DBA restores the **student database from the latest backup**.

3. Database Performance Management

The DBA monitors and improves database performance by optimizing queries, indexes, and storage.

Example: The DBA creates an **index on the PRN field** to make student record searches faster.

5. Database Maintenance:

The DBA performs regular maintenance such as updating database software, managing storage, and reorganizing database files.

Example: The DBA removes unnecessary temporary data and reorganizes storage to maintain efficient database operation.

6. Data Integrity and Availability:

The DBA ensures that data remains **accurate, consistent, and available** to authorized users.

Example: The DBA ensures that each student's **PRN is unique** and that the database remains available while examination results are being processed.